

Section 1: Core Java & Object-Oriented Design

Q: What is the difference between `==` and `.equals()` in Java?

A:

`==` compares object references (whether they point to the same object).

`.equals()` compares values (if the class overrides it).

```
String a = new String("Hi");
String b = new String("Hi");
System.out.println(a == b);      // false
System.out.println(a.equals(b)); // true
```

Follow-up: Always override both `equals()` and `hashCode()` when using objects in collections like `HashMap`.

Q: What's the difference between `ArrayList` and `LinkedList`?

A:

- `ArrayList` is backed by a dynamic array (fast random access, slow insert/delete).
- `LinkedList` is a doubly-linked list (fast insert/delete, slow random access).

Follow-up: Use `ArrayList` for lookups and `LinkedList` for frequent inserts/removals.

Q: How does Garbage Collection (GC) work?

A:

- GC automatically frees unused memory by removing unreachable objects.
- Key areas: Eden, Survivor, and Old Generation.
- Use `-XX:+PrintGCDetails` for GC logs and tune with G1GC or ZGC.

Follow-up: Finalization is deprecated; prefer `try-with-resources`.

💬 *“Clarity in basics is your silent confidence booster.”*

Section 2: Java Concurrency & Multithreading

Q: How do you create and manage threads using `ExecutorService`?

A:

Use a thread pool for efficient resource management:

```
ExecutorService executor = Executors.newFixedThreadPool(4);
for (int i = 0; i < 10; i++) {
    executor.submit(() -> System.out.println(Thread.currentThread().getName()));
}
executor.shutdown();
```

Follow-up: Always shut down the executor to release threads gracefully.

Q: Difference between `ReentrantLock` and `synchronized`?

A:

- `ReentrantLock` gives more control (tryLock, fairness, interruptible locks).
- `synchronized` is simpler but implicit.

```
Lock lock = new ReentrantLock();
try {
    lock.lock();
    // critical section
} finally {
    lock.unlock();
}
```

Q: How does `Semaphore` help limit concurrency?

A:

```
Semaphore sem = new Semaphore(3);
sem.acquire();
try {
    // only 3 threads at a time
} finally {
    sem.release();
}
```

Useful in connection pools and rate limiting.

Q: How do `CompletableFuture` improve async programming?

A:

Enables non-blocking async code:

```
CompletableFuture.supplyAsync(() -> fetchUser())
    .thenApply(this::enrich)
    .thenAccept(System.out::println);
```

Avoids callback hell and enables parallel composition.

Q: What are Virtual Threads (Java 21)?

A:

They're lightweight threads managed by the JVM, scaling to thousands efficiently:

```
try (var exec = Executors.newVirtualThreadPerTaskExecutor()) {
    IntStream.range(0, 10).forEach(i ->
        exec.submit(() -> System.out.println(Thread.currentThread())));
}
```

💬 “Every concurrency bug you fix sharpens your discipline — mastery builds thread by thread.”

Section 3: Spring & System Design

Q: What is Dependency Injection (DI) in Spring?

A:

Injects dependencies rather than creating them manually.

```
@Service
public class OrderService {
    private final PaymentService paymentService;
    @Autowired
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

DI allows objects to depend on abstractions rather than concrete implementations.

Follow-up: Spring manages dependencies via the `ApplicationContext` and annotations like `@Autowired`, `@Primary`, and `@Qualifier`.

Q: What's the difference between `@Component`, `@Service`, and `@Controller`?

A:

All are stereotypes marking Spring-managed beans.

- `@Component`: generic bean
 - `@Service`: business logic layer
 - `@Controller`: web layer handling HTTP requests
-

Q: How do you secure REST APIs using JWT in Spring Security?

A:

Use filters to intercept requests, validate JWT, and populate the security context.

Follow the **JWT** → **Authentication Filter** → **SecurityContext** pipeline.

Q: What is the CAP theorem?

A:

Distributed systems can only guarantee two of three:

- Consistency
- Availability
- Partition Tolerance

💬 *“Designs evolve. Principles stay constant. Trust the process.”*

Section 4: Kafka, Messaging & Event-Driven Architecture

Q: What is a Kafka Partition & Consumer Group?

A:

- Each topic is divided into partitions.
 - Consumers in a group share partitions — ensures parallel processing.
-

Q: Why is Kafka fast?

A:

- Sequential disk writes (log-based).
 - Zero-copy data transfer.
 - Batch compression and offset-based fetching.
-

Q: Explain Zookeeper's role in Kafka.

A:

- Manages broker metadata and leader election (being phased out in KRaft mode).
-

Q: What is a Dead Letter Queue (DLQ)?

A:

- Stores messages that failed processing after retries — prevents loss and aids debugging.
-

💬 *"Reliable systems are born from consistent failure handling."*

Section 5: Design Patterns & Java Architecture

Q: Explain the Strategy Pattern.

A:

Interchangeable behaviors encapsulated via an interface.

```
interface PaymentStrategy { void pay(int amount); }
class CreditCard implements PaymentStrategy { public void pay(int amount) { ... } }
class PaymentService {
    private final PaymentStrategy strategy;
    public PaymentService(PaymentStrategy strategy) { this.strategy = strategy; }
}
```

Q: Factory Pattern example?

A:

Centralizes object creation.

```
public class ShapeFactory {
    public Shape getShape(String type) {
        return switch(type) {
            case "CIRCLE" -> new Circle();
            case "RECT" -> new Rectangle();
            default -> throw new IllegalArgumentException();
        };
    }
}
```

Q: Prototype vs Singleton?

A:

- Prototype → new instance each time.
 - Singleton → single shared instance.
-

💬 *“Patterns are wisdom made repeatable — don’t memorize, internalize.”*

Section 6: Database Consistency & SAGA Pattern

Q: What is the SAGA pattern?

A:

Manages distributed transactions using compensating actions for rollback.

- **Choreography** → event-driven.
 - **Orchestration** → central coordinator.
-

Q: How do you ensure idempotency in distributed systems?

A:

Use unique transaction IDs, versioning, and upsert operations to prevent duplicates.

Q: What is Eventual Consistency?

A:

Data syncs asynchronously across systems but converges over time.

💬 *"Consistency isn't instant — it's trust built over retries."*

Section 7: JVM Performance & Profiling

Q: How does JIT optimization work?

A:

JIT compiles hot methods at runtime for better performance.

Q: How to tune GC?

A:

Use G1GC or ZGC; monitor pauses with `-Xlog:gc` and tune heap ratios.

Q: Profiling tools?

A:

- `jvisualvm`, `jcmd`, `jstack`, `async-profiler`
 - Thread dumps reveal blocked threads and deadlocks.
-

💬 *"Profiling isn't about speed — it's about understanding your system's rhythm."*

Section 8: Behavioral & Motivational Reflections

Q: How do you handle pressure during interviews?

A:

- Pause before answering — clarity > speed.
 - Rephrase the question to confirm understanding.
 - Smile — it resets your brain's stress response.
-

Q: How to handle a question you don't know?

A:

Acknowledge it, reason through what you *do* know, and show structured thinking.

Q: What to do if you make a silly mistake?

A:

Own it calmly, correct it, and keep momentum. Most interviewers value composure over perfection.

💬 *"You're already prepared — now, just be present."*
